

→JS-6

→Functions in JS

Functions in JS

Function Definition (telling JS)

```
function funcName( ) {  
    //do something  
}
```

```
function hello() {  
    console.log("hello");  
}
```

Function Calling (Using the function)

```
funcName( );
```

```
hello();
```

//function with Arguments

Functions with Arguments

Values we pass to the function

```
function funcName(arg1, arg2.. ) {  
    //do something  
}
```

→Return keyword

Return

return keyword is used to return some value from the function.



```
function funcName(arg1, arg2.. ) {  
  //do something  
  return val;  
}
```

```
function isAdult(age) {  
  if(age >= 18) {  
    return "adult";  
  } else {  
    return "not adult";  
  }  
  console.log("bye bye");  
}
```

→ whatever we write after the return statement it will not execute anything

→ what is a scope

Scope

Scope determines the **accessibility** of variables, objects, and functions from different parts of the code.

- Function
- Block
- Lexical

→Function scope

Function Scope

Variables defined inside a function are not accessible (visible) from outside the function.

```
let sum = 54; //Global Scope

function calSum(a, b) {
  let sum = a+b; //Function Scope
  console.log(sum);
}

calSum(1, 2);
```

→in both the function scope and the global scope in function the function scope sum will be only worked

→block scope

Block Scope

Variables declared inside a { } block cannot be accessed from outside the block.

<pre>{ const a = 25; } console.log(a);</pre>	<pre>{ var a = 25; } console.log(a);</pre>
---	---

→Lexical scope:

Lexical Scope

→ nested function

a variable defined outside a function can be accessible inside another function defined after the variable declaration.

The opposite is NOT true.

```
function outerFunc() {
  let x = 5;
  let y = 6;
  function innerFunc() {
    console.log(x);
  }
  innerFunc();
}
```

```
function outerFunc() {
  function innerFunc() {
    console.log(x);
    console.log(y);
  }
  let x = 5;
  let y = 6;
  innerFunc();
}
```

Practice Qs

What will be the output?

```
let greet = "hello";

function changeGreet() {
  let greet = "namaste";
  console.log(greet);
  function innerGreet() {
    console.log(greet);
  }
}

console.log(greet);
changeGreet();
```

["hello"
"namaste"]

greet = "hello" // Global
greet = "namaste" // Function

jcreddy2096@g

```
let greet = "hello"; //global scope

function changeGreet() {
  let greet = "namaste"; //function scope
  console.log(greet);

  function innerGreet() {
    console.log(greet); //lexical scope
  }

  innerGreet();
}

console.log(greet);
changeGreet();
```

→Function Expressions

Function Expressions

```
const variable = function(arg1, arg2..) {  
  //do or return something  
}
```

```
const sum = function(a, b) {  
  return a + b;  
}  
  
sum(2, 3);
```

→takes one or multiple functions as arguments

Higher Order Functions

Takes one or multiple functions as arguments

```
function multipleGreet(func, n) {  
  for(let i=1; i<=n; i++) {  
    func();  
  }  
}
```

idyl2096@gmail.com

```
let greet = function() {  
  console.log("hello");  
}
```

```
multipleGreet(greet, 2);
```

2 hello

→ Higher order functions returns a function

Higher Order Functions

Returns a function

```
function oddEvenTest(request) {  
  if(request == "odd") {  
    return function(n) {  
      console.log(!(n%2 == 0));  
    }  
  } else if(request == "even") {  
    return function(n) {  
      console.log(n%2 == 0);  
    }  
  } else {  
    console.log("wrong request");  
  }  
}
```

→ Methods

Methods

Actions that can be performed on an object.

```
const calculator = {  
  add: function(a, b) {  
    return a + b;  
  },  
  sub: function(a, b) {  
    return a - b;  
  },  
  mul: function(a, b) {  
    return a * b;  
  }  
};
```

→JS-7

→this keyword

this Keyword

“This” keyword refers to an object that is executing the current piece of code.

```
app.js / student / getAvg
const student = {
  name: "shradha",
  age: 23,
  eng: 95,
  math: 93,
  phy: 97,
  getAvg() {
    console.log(this);
    let avg = (this.eng + this.math + this.phy) / 3;
    console.log(`${this.name} got avg marks = ${avg}`);
  }
}
```

top Filter Default levels No Issues

```
> student.getAvg();
  {name: 'shradha', age: 23, eng: 95, math: 93, phy: 97, _} app.js:8
shradha got avg marks = 95 app.js:10
< undefined
```

→we made an function outside the class and see what it will give output

```

app.js > getAvg
1  const student = {
2      name: "shradha",
3      age: 23,
4      eng: 95,
5      math: 93,
6      phy: 97,
7      getAvg() {
8          console.log(this);
9          let avg = (this.eng + this.math + this.phy) / 3;
10         console.log(`${this.name} got avg marks = ${avg}`);
11     }
12 }
13
14 function getAvg() {
15     console.log(this);
16 }

```

```

> getAvg();
app.js:15
Window {window: Window, self: Window, document: document, name: '', l
ocation: Location, ...}
< undefined
>

```

shady2096@gmail.com

File | /Users/shradhakapra/WebClassroom/JavaScript/index.html

Elements Console Sources This page says

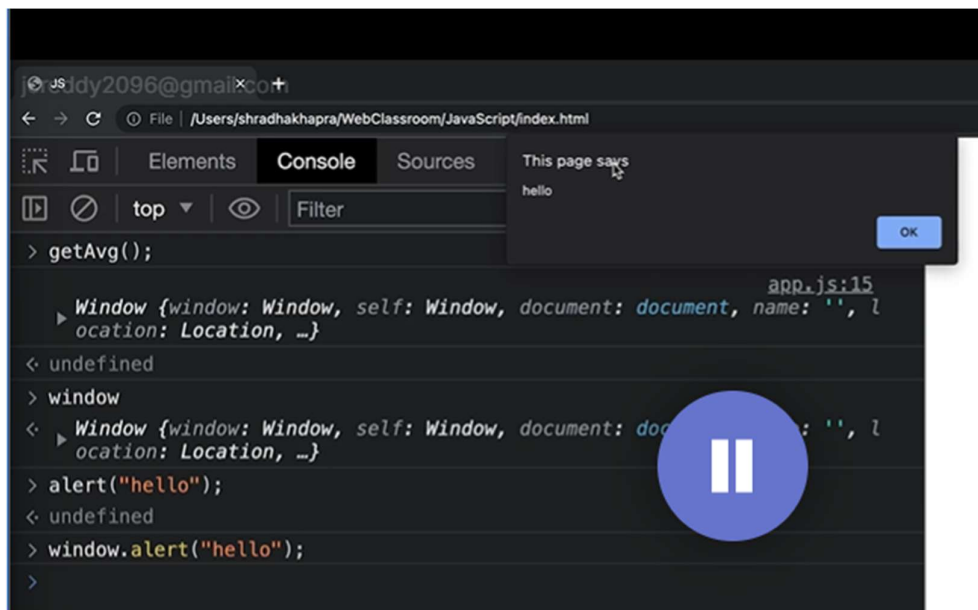
hello

OK

```

> getAvg();
app.js:15
Window {window: Window, self: Window, document: document, name: '', l
ocation: Location, ...}
< undefined
> window
Window {window: Window, self: Window, document: document, name: '', l
ocation: Location, ...}
> alert("hello");

```



→try and catch

try & catch

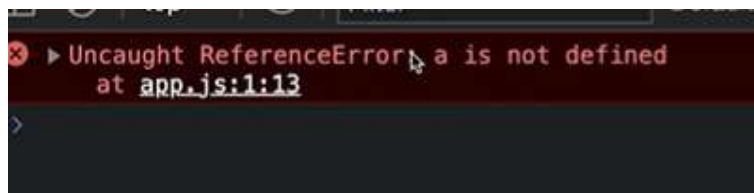
The try statement allows you to define a block of code to be tested for errors while it is being executed.

The catch statement allows you to define a block of code to be executed, if an error occurs in the try block.

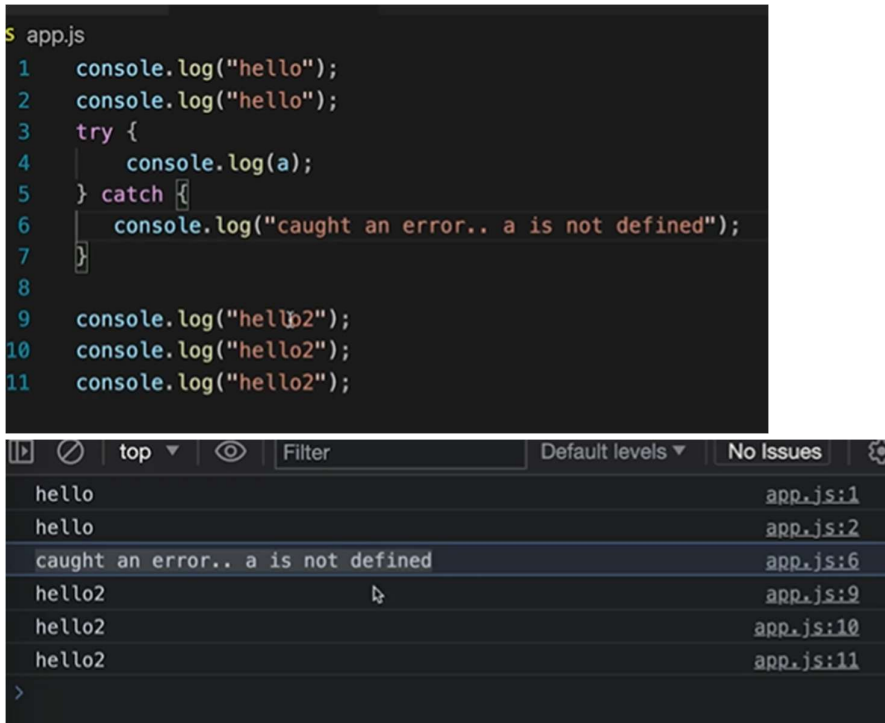
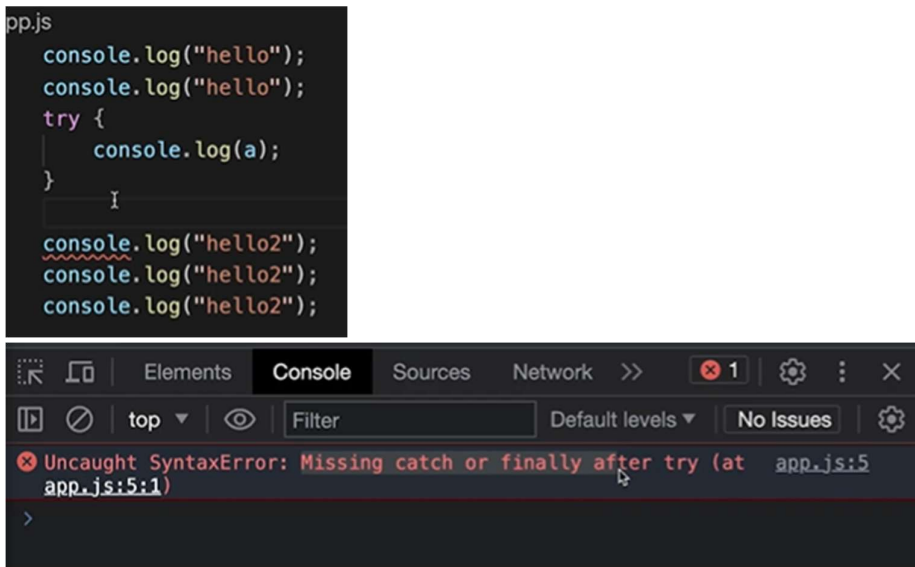
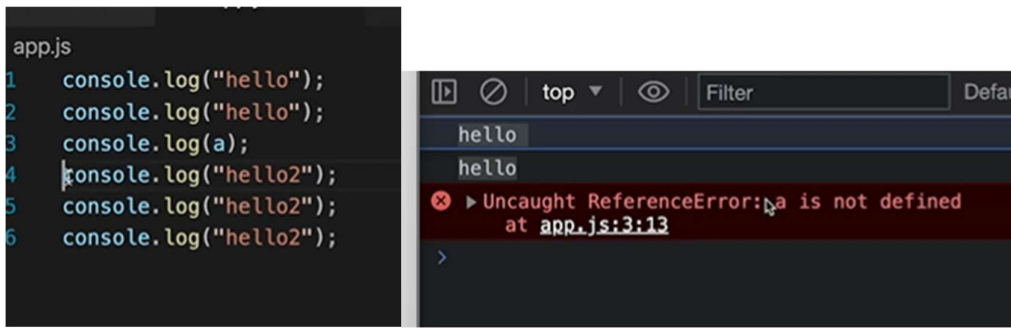
```
try {  
  console.log(a);  
} catch {  
  console.log("variable a doesn't  
}
```

→this error comes when we didn't define the variable

```
app.js  
1 console.log(a);
```



→when we execute the below program the code get executed only upto 2nd hello because after that it will get error because we didn't declare the a.



```

app.js > ...
  console.log("hello");
  console.log("hello");
  let a = 5;
  try {
    console.log(a);
  } catch {
    console.log("caught an error.. a is not defined");
  }

  console.log("hello2");
  console.log("hello2");
  console.log("hello2");

```

```

hello
hello
5
hello2
hello2
hello2

```

```

1  console.log("hello");
2  console.log("hello");
3  // let a = 5;
4  try {
5    console.log(a);
6  } catch(err) {
7    console.log("caught an error.. a is not defined");
8    console.log(err);
9  }
10
11 console.log("hello2");
12 console.log("hello2");
13 console.log("hello2");

```

```

hello
hello
caught an error.. a is not defined
ReferenceError: a is not defined
  at app.js:5:17
hello2
hello2
hello2

```

→ Arrow functions

Arrow Functions

`const func = (arg1, arg2 ..) => { function definition }`

```
const sum = (a, b) => {  
  console.log(a+b);  
}
```

app.js > [x] sum

```
const sum = (a, b) => {  
  console.log(a + b);  
};
```

No Issues

> sum(2, 3);

5

< undefined

< undefined

> sum

< (a, b) => {
 console.log(a + b);
}

>

```
const cube = (n) => {  
  return n * n * n;  
};
```

No Issues

> cube(2);

< 8

```
const pow = (a, b) => {  
  return a ** b;  
};
```

No Issues

> pow(2, 4);

< 16

>

→ in arrow functions for a single argument we can execute without the parentheses also we can execute

```
> const cube = n => {  
  return n * n * n;  
};  
  
< undefined
```

```
const hello = () => {  
  console.log("hello world");  
};  
  
> hello();  
hello world  
< undefined  
>
```

→ implicit return arrow functions

Arrow Functions

Implicit return

① return ✓
print x
calculator

```
const func = (arg1, arg2 ..) => { value }
```

```
const mul = (a, b) => (  
  a * b  
);
```

```
const mul = (a, b) => a * b;
```

```
> mul(2, 3);  
< 6  
> mul(2, 19);  
< 38
```

```
const sum = (a, b) => a + b;
sum(4, 5);
9
```

→ Set Timeout

Set Timeout

setTimeout(function, timeout)

```
console.log("hi there!");

setTimeout( () => {
  console.log("Apna College");
}, 4000);

console.log("welcome to");
```

```
app.js > setTimeout() callback
1 console.log("Hi there!");
2
3 setTimeout(() => {
4   console.log("Apna College");
5 }, 4000);
6
7 console.log("Welcome to");
8
```

No Issues

Hi there!

Welcome to

Apna College

>

→ Set Interval

Set Interval

`setInterval( function, timeout )`

```
setInterval( () => {  
  console.log("Apna College");  
}, 2000);
```

`clearInterval( id )`

Set Interval

`setInterval( function, timeout )`
Interval

```
setInterval( () => {  
  console.log("Apna College");  
}, 2000);
```

`clearInterval( id )`

time (ms)

time
↓
exec

time
↓
exec
↓ *time*
exec
↓ *time*
exec

jcreddy2096@gm

6 Apna College

>

```
setInterval(() => {  
  console.log("Apna College");  
}, 2000);
```

```
let id = setInterval(() => {  
  console.log("Apna College");  
}, 2000);  
  
console.log(id);
```

1

```

Hello World
2 Apna College
Hello World
Apna College
Hello World
2 Apna College
Hello World
Apna College
Hello World
2 Apna College
Hello World
Apna College
Hello World
Apna College
> clearInterval(id);
< undefined
4 Hello World
> clearInterval(id2);
< undefined
>

```

this with Arrow Functions

Arrow
 1) lexical scope
 parent → call scope

function
 1) scope → this → calling object

```

op.js > [?] student
const student = {
  name: "aman",
  marks: 95,
  prop: this, //global scope
};

const a = 5; //global scope

```

```

student
▼ {name: 'aman', marks: 95, prop: Window} ⓘ
  marks: 95
  name: "aman"
  ▶ prop: Window {window: Window, self: Window, document: do
  ▶ [[Prototype]]: Object

```

```

app.js > [?] student > [?] getName
const student = {
  name: "aman",
  marks: 95,
  prop: this, //global scope
  getName: function () {
    console.log(this);
    return this.name;
  },
};

```

```

> student.getName()
app.js:6
{name: 'aman', marks: 95, prop: Window, getName: f}
  getName: f()
  marks: 95
  name: "aman"
  prop: Window {window: Window, self: Window, document: do
  [[Prototype]]: Object
< 'aman'
>

```

→ arrow functions this

```

app.js > [?] student
const student = {
  name: "aman",
  marks: 95,
  prop: this, //global scope
  getName: function () {
    console.log(this);
    return this.name;
  },
  getMarks: () => {
    console.log(this); //parent's scope
    return this.marks;
  },
};

```

```

No Issues
> student.getName()
app.js:6
{name: 'aman', marks: 95, prop: Window, getName: f, getMa
  rks: f}
< 'aman'
> student.getMarks()
app.js:10
Window {window: Window, self: Window, document: document,
  name: '', location: Location, ...}
< undefined
>

```

```

    },
    getInfo1: function () {
      setTimeout(() => {
        console.log(this); //student
      }, 2000);
    },
    getInfo2: function () {
      setTimeout(function () {
        console.log(this); //window
      }, 2000);
    },
  };
};

```

```

> student.getInfo1();
< undefined

app.js:15
  {name: 'aman', marks: 95, prop: Window, getName: f, getMa
    rks: f, ...}
> student.getInfo2();
< undefined

app.js:20
  Window {window: Window, self: Window, document: document,
    name: '', location: Location, ...}
>

```